Manual de instalación

Sistema de reserva de turnos | GrupoIIS2026 | Parte 6
Versión 1.0 · 6 de noviembre de 2026

Objetivo: instalar el producto desde sus fuentes, levantar sus siete servicios y comprobar la conexión HTTP, MQTT y MySQL. Los comandos de este manual se ejecutan en Windows PowerShell. Cada bloque contiene comandos completos; ejecutar las líneas de comandos por separado.

1. Requisitos previos

Disponer de Docker Desktop instalado, con el motor de contenedores Linux iniciado y Docker Compose disponible mediante docker compose. Se necesita conexión a Internet para la primera descarga de imágenes y dependencias Maven. Tener el proyecto completo extraído en una carpeta local y permisos para ejecutar Docker.

PowerShell y curl.exe se utilizan para las comprobaciones. MySQL Workbench y MQTT Explorer son opcionales. Java 21 y Maven se ejecutan dentro de las imágenes de compilación: no es necesario instalarlos localmente para iniciar el producto con Docker.

2. Archivos que deben conservarse

Ruta en el proyecto	Uso
docker-compose.yml	Define todos los servicios, puertos, red y volumen MySQL.
Dockerfile, pom.xml y src/	Construcción de los procesos Java principales.
api-rest/	Módulo de la API, con Dockerfile, pom.xml y src/.
mosquitto/config/mosquitto.conf	Configuración montada en Mosquitto.
scripts/parte6/	Scripts de operaciones HTTP y consultas SQL.
documentacion/parte6/	Manuales, diagramas y especificaciones de la entrega.

La configuración MQTT de demostración contiene listener 1883 y allow_anonymous true. Las carpetas mosquitto/data y mosquitto/log se usan como montajes del broker; conservarlas o permitir que Docker las cree. No ejecutar el producto directamente dentro del ZIP.

3. Puertos utilizados

Puerto en la computadora	Destino
8080	API REST: http://localhost:8080/api
1883	Broker MQTT: localhost:1883
3307	MySQL externo: localhost:3307; dentro de la red: mysql-db:3306

Los puertos y los nombres fijos de contenedores deben estar libres. Si hay otra copia del proyecto ejecutándose, detenerla desde su carpeta antes de iniciar esta instalación.

Primer arranque y verificación

4. Preparar la terminal

Abrir Docker Desktop y esperar a que el motor esté disponible. Abrir PowerShell en la carpeta que contiene docker-compose.yml. Si se usa este ejemplo, sustituir la ruta por la carpeta real:

```
cd "C:\ruta\Taller-Ingenieria-de-Software-main"
docker info
docker compose config
```

docker info debe mostrar información del servidor sin errores de conexión. docker compose config debe imprimir una configuración válida, sin errores YAML.

5. Construir e iniciar

```
docker compose up -d --build
docker compose ps -a
```

La primera construcción puede demorar mientras descarga imágenes y dependencias. Los Dockerfiles compilan y empaquetan los JAR con Maven y ejecutan Java 21. La construcción actual usa -DskipTests; un build exitoso no equivale a ejecutar los tests.

Se esperan los servicios mysql-db, mosquitto, api-rest, app-listener, app-generador, procesador-solicitudes y facturador. MySQL debe figurar healthy y los procesos deben permanecer activos. El healthcheck MySQL ejecuta SELECT 1 por TCP. El estado Up de una aplicación no reemplaza la revisión de sus registros.

6. Comprobar inicialización

```
docker compose logs --tail 20 app-listener procesador-solicitudes facturador
docker compose exec -T mysql-db mysql -uroot -proot -D turnosdb -e "SHOW TABLES;"
```

Se esperan las tablas CLIENTE, ESTABLECIMIENTO, FACTURA, ITEMFACTURA, PERSONAL y RESERVATURNO. En una base vacía, el listener carga tres establecimientos y tres empleados de prueba. Si ya existe personal, omite esa carga. El generador puede comenzar a crear reservas durante la comprobación.

```
curl.exe -i http://localhost:8080/api/establecimientos
```

Resultado esperado: HTTP 200 y una lista JSON. En una instalación nueva se observaron tres establecimientos y tres empleados. El listener debe indicar conexión a Mosquitto y guardar reservas; el procesador debe evaluar SOLICITADO. El facturador ejecuta ciclos aun si no hay turnos para facturar.

Accesos adicionales: Swagger UI en http://localhost:8080/api/swagger-ui.html y definición generada en http://localhost:8080/api/api-docs. La especificación revisada de la entrega se encuentra en documentacion/parte6/especificacion-api.json; sus mejoras documentales no cambian automáticamente el Swagger generado.

Configuración y operación

7. Configuración del entorno

Configuración	Valor y alcance
MySQL externo	Host 127.0.0.1; puerto 3307; base turnosdb; usuario root; contraseña root. Son credenciales de demostración.
BROKER_URL	tcp://mosquitto:1883 en los procesos Docker.
DB_URL / DB_USER / DB_PASSWORD	La API los recibe de Compose. Los procesos principales usan persistence.xml para su conexión JPA.
TZ	America/Montevideo en las aplicaciones. Las fechas de turno se manejan sin offset.
VALIDATION_INTERVAL_SECONDS	60 en procesador-solicitudes.
BILLING_INTERVAL_SECONDS	300 en facturador, equivalente a cinco minutos.
MAIN_CLASS	Selecciona la clase Java de cada proceso del módulo principal.

Si se modifican credenciales, se debe mantener coherencia entre Compose, configuración JPA y API. Cambiar MYSQL_ROOT_PASSWORD en Compose no cambia automáticamente la contraseña de una base ya inicializada.

8. Detener, reanudar y reconstruir

```
docker compose stop
docker compose start
```

stop detiene los contenedores; start reanuda los existentes. Para retirar contenedores y red conservando la base, y luego recrearlos:

```
docker compose down
docker compose up -d
```

El volumen mysql_data conserva MySQL. No agregar -v a down si se desean conservar los datos. Los montajes del broker se conservan en las carpetas locales. Después de cambiar código del listener, por ejemplo:

```
docker compose up -d --build app-listener
```

Para detener solo la simulación de clientes sin detener la API ni el procesamiento:

```
docker compose stop app-generador
docker compose start app-generador
```

9. Scripts de comprobación entregados

Desde la raíz del proyecto, ejecutar el script requerido; las pruebas de operaciones crean y modifican datos de demostración:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\parte6\establecimientos.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\parte6\personal.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\parte6\reservas.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\parte6\consultar-base.ps1
```

Diagnóstico y prueba con base nueva

10. Problemas frecuentes

Síntoma	Acción
No existe dockerDesktopLinuxEngine	Abrir Docker Desktop y esperar al motor. Verificar docker info; si estaba abierto, reiniciar Docker Desktop.
Error YAML / expected key	Revisar espacios e indentación del Compose. Ejecutar docker compose config.
Nombre de contenedor ocupado	Identificar con docker ps -a qué proyecto usa ese nombre. Detener el proyecto correspondiente antes de iniciar otra copia.
Puerto ocupado	Detener la aplicación que lo utiliza o ajustar el puerto externo de Compose y los comandos del cliente.
Connection refused / Exited / Restarting	Revisar docker compose ps -a y docker compose logs --tail 80 SERVICIO, sustituyendo SERVICIO por el nombre afectado. Confirmar MySQL healthy y Mosquitto disponible antes de reiniciar el servicio.
Duplicate foreign key constraint name	EclipseLink intenta extender un esquema que ya contiene la restricción. Se observaron estas advertencias sin impedir el funcionamiento. No borrar tablas para resolverlas; revisar actividad posterior y configuración de generación del esquema.
Ciclo de facturacion: 0	No encontró turnos ATENDIDO pendientes en ese ciclo. Por sí solo no indica una falla.
mysql: warning sobre contraseña	El cliente avisa que se pasó la contraseña por línea de comandos. Revisar la salida SQL para saber si la consulta se completó.

11. Prueba aislada de MySQL desde cero

Para repetir la prueba verificada, retirar el proyecto original sin borrar volúmenes y utilizar otro nombre de proyecto:

```
docker compose down
docker compose -p parte6-prueba up -d --build
docker compose -p parte6-prueba ps -a
```

Con el Compose entregado, parte6-prueba crea su propio volumen MySQL. Durante esa prueba se debe usar -p parte6-prueba en todos los comandos de Compose. La configuración y las carpetas de Mosquitto siguen siendo compartidas: se aísla la base, no todos los archivos del producto. No ejecutar ambas copias simultáneamente por sus nombres y puertos fijos.

Volver al proyecto original conservando ambas bases:

```
docker compose -p parte6-prueba down
docker compose up -d
```

Validación realizada: arranque con base nueva, seis tablas, datos iniciales, respuesta HTTP 200 y procesamiento de reservas. Al volver al proyecto original se verificó la conservación de las reservas 7 y 9 en FACTURADO. La prueba fue ejecutada en el entorno Windows del equipo; no supone una certificación para otros entornos.